

SMART PARKING SYSTEM USING ARDUINO

Link:- <https://www.tinkercad.com/things/dqUaD53YcT7-smart-parking-slot-detection-system?sharecode=L0-fwXPohBBfjkgpuzVN078-5QP-AftxAjqktvlwk0>

1. INTRODUCTION

Finding an available parking space can be difficult in crowded areas such as shopping centers, offices, educational institutions, hospitals, and public parking facilities.

A smart parking system can solve this problem by continuously monitoring parking spaces and informing users about available locations.

This project implements a basic smart parking system using an **Arduino**, four switches/sensors, eight LEDs, and an I2C LCD display.

Each parking slot has two indicators:

- **Green LED:** Slot is vacant.
- **Red LED:** Slot is occupied.

The Arduino reads the status of all four parking slots, processes the information, calculates the number of vacant slots, and displays the result on the LCD.

2. OBJECTIVES

The main objectives of this project are:

1. To design a simple automated parking-slot monitoring system.
2. To detect whether each parking slot is occupied or vacant.
3. To provide visual status indication using red and green LEDs.
4. To display the number of vacant slots on a 16×2 LCD.
5. To display “**PARKING FULL**” when all four slots are occupied.
6. To display individual slot information through the Arduino Serial Monitor.
7. To understand the practical application of Arduino digital input/output.
8. To develop a foundation for a larger smart parking system.

3. COMPONENTS REQUIRED

S. No.	Component	Quantity	Purpose
1	Arduino board	1	Main controller

2	16×2 I2C LCD	1	Displays parking information
3	Parking switches/sensors	4	Detect slot occupancy
4	Red LEDs	4	Indicate occupied slots
5	Green LEDs	4	Indicate vacant slots
6	Resistors	8	LED current limiting
7	Breadboard	1	Circuit assembly
8	Jumper wires	As required	Connections
9	USB cable/power supply	1	Arduino power/programming

4. HARDWARE DESCRIPTION

4.1 Arduino

The Arduino acts as the main controller of the system. It reads the four parking-slot inputs and controls the eight LEDs and LCD.

The program continuously checks the status of each parking slot.

4.2 Parking Slot Switches/Sensors

Four inputs are used to represent four parking slots:

- Slot 1 → A0
- Slot 2 → A1
- Slot 3 → A2
- Slot 4 → A3

The program uses the Arduino's internal pull-up resistors through:

`INPUT_PULLUP`

Therefore:

- **LOW = Switch pressed = Slot occupied**
- **HIGH = Switch released = Slot vacant**

Actual parking sensors such as ultrasonic, IR, or magnetic sensors can replace the switches without changing the basic concept of the system.

4.3 Red LEDs

Each parking slot has one red LED.

Slot Red LED Pin

6
8
10
12

A red LED indicates that the corresponding parking slot is occupied.

4.4 Green LEDs

Each parking slot also has one green LED.

Slot Green LED Pin

7
9
11
13

A green LED indicates that the corresponding parking slot is vacant.

4.5 16×2 I2C LCD

The LCD is used to display the number of vacant parking slots.

The code initializes the LCD with:

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

The I2C address used in the program is **0x27**.

Example display:

Vacantslots:2

Thank You

When all slots are occupied:

Vacantslots:0
PARKING FULL

5. PIN CONFIGURATION

Device	Arduino Pin
Slot 1 sensor/switch	A0
Slot 2 sensor/switch	A1
Slot 3 sensor/switch	A2
Slot 4 sensor/switch	A3
Slot 1 Red LED	D6
Slot 1 Green LED	D7
Slot 2 Red LED	D8
Slot 2 Green LED	D9
Slot 3 Red LED	D10
Slot 3 Green LED	D11
Slot 4 Red LED	D12
Slot 4 Green LED	D13
LCD	I2C

6. WORKING PRINCIPLE

The system operates continuously in a loop.

Step 1: Read Sensors

The Arduino reads the status of all four switches:

- A0 → Slot 1

- A1 → Slot 2
- A2 → Slot 3
- A3 → Slot 4

A LOW reading means that the slot is occupied.

Step 2: Control LEDs

For every parking slot, the Arduino controls one red and one green LED.

If the slot is occupied:

- Red LED = ON
- Green LED = OFF

If the slot is vacant:

- Red LED = OFF
- Green LED = ON

Step 3: Calculate Available Slots

The program starts with:

occupied = 0

For every occupied slot, the value of occupied is increased by one.

The number of available slots is calculated as:

available = 4 - occupied

Step 4: Update LCD

The LCD displays the number of vacant slots.

For example, if two slots are occupied:

Vacant slots: 2

The second line displays:

Thank You

If all four slots are occupied:

Vacant slots: 0

and:

PARKING FULL

Step 5: Serial Monitor

The Arduino also sends the status of every slot to the Serial Monitor at 9600 baud.

Example:

Slot 1: OCCUPIED

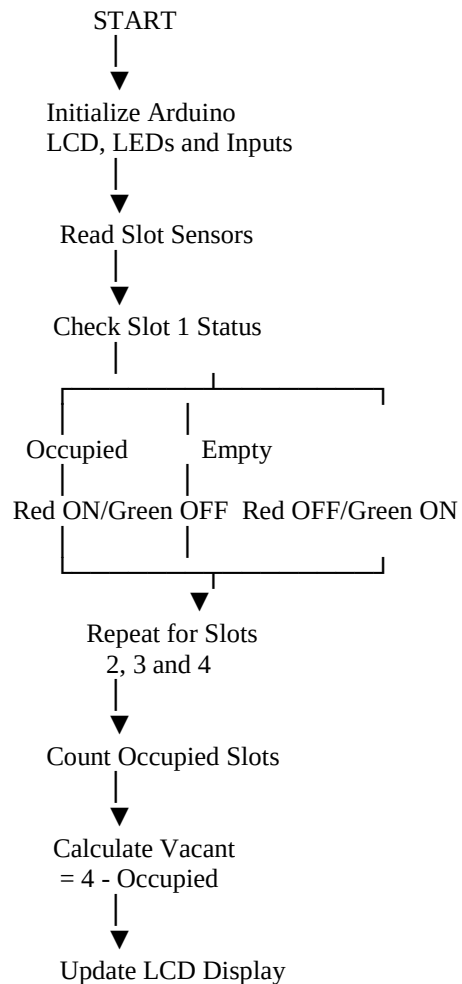
Slot 2: EMPTY

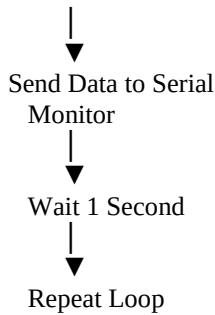
Slot 3: EMPTY

Slot 4: OCCUPIED

Available Slots: 2

7. FLOWCHART





8. SOFTWARE REQUIREMENTS

The software required for this project includes:

- Arduino IDE
- Arduino programming language based on C/C++
- Wire.h library
- LiquidCrystal_I2C.h library

The Wire.h library provides I2C communication, while the LiquidCrystal_I2C.h library provides control of the I2C LCD.

9. PROGRAM DESCRIPTION

The program begins by including the required libraries:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

The LCD object is initialized using:

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

The four parking inputs are assigned to A0–A3.

The red and green LEDs are assigned to digital pins 6–13.

In setup(), the parking inputs are configured using:

```
pinMode(SLOT1, INPUT_PULLUP);
```

Similar configuration is performed for all four slots.

The LED pins are configured as outputs.

The LCD is initialized using:

```
lcd.init();  
lcd.backlight();
```

The Serial Monitor is initialized at:

```
Serial.begin(9600);
```

The startup message displayed on the LCD is:

```
SMART PARKING  
System Ready
```

After two seconds, the LCD is cleared and normal operation begins.

11. SENSOR LOGIC

The sensor logic is based on the Arduino internal pull-up resistor.

The following statement determines whether Slot 1 is occupied:

```
bool slot1Occupied = (digitalRead(SLOT1) == LOW);
```

The same logic is used for the remaining slots.

Therefore:

Sensor State	Arduino Input	Parking Status
--------------	---------------	----------------

Switch pressed	LOW	Occupied
----------------	-----	----------

Switch released	HIGH	Vacant
-----------------	------	--------

This arrangement reduces the need for external pull-up resistors for the switches.

11. LED LOGIC

The LED control follows a simple conditional structure.

For an occupied slot:

```
digitalWrite(RED1, HIGH);  
digitalWrite(GREEN1, LOW);
```

For a vacant slot:

```
digitalWrite(RED1, LOW);
```

```
digitalWrite(GREEN1, HIGH);
```

The same operation is performed for all four slots.

12. LCD OUTPUT

The first LCD line displays the number of vacant slots:

```
lcd.print("Vacant slots: ");  
lcd.print(available);
```

When no parking space is available, the second line displays:

PARKING FULL

Otherwise it displays:

Thank You

13. SERIAL MONITOR OUTPUT

The Serial Monitor provides detailed information about the parking system.

At every cycle, it reports:

- Slot 1 status
- Slot 2 status
- Slot 3 status
- Slot 4 status
- Number of available slots

The communication speed is **9600 baud**.

This feature is useful for debugging and monitoring the system during development.

14. TESTING AND RESULTS

The system can be tested by changing the state of each parking-slot switch.

Slot 1 Slot 2 Slot 3 Slot 4 Vacant Slots LCD Status

4

3

2

1

0

The corresponding LEDs should also change according to the parking status.

For example, when Slot 1 is occupied:

- Slot 1 red LED → ON
- Slot 1 green LED → OFF

When Slot 1 is vacant:

- Slot 1 red LED → OFF
- Slot 1 green LED → ON

15. ADVANTAGES

The proposed system has several advantages:

1. Simple and easy to construct.
2. Low-cost hardware implementation.
3. Real-time parking-slot monitoring.
4. Clear visual indication using red and green LEDs.
5. LCD provides the total number of vacant spaces.
6. Serial Monitor provides detailed diagnostic information.
7. Easy to expand to additional parking slots.
8. Arduino programming makes the system easy to modify.
9. Suitable for educational and prototype applications.
10. Can be upgraded to use automatic parking sensors.

16. LIMITATIONS

The current prototype has some limitations:

1. It supports only four parking slots.
2. Push buttons/switches are used instead of actual vehicle-detection sensors.
3. It does not automatically open or close a parking gate.
4. It does not provide a mobile or web-based interface.
5. There is no remote monitoring facility.
6. The LCD only displays the total number of vacant slots rather than guiding the driver to a specific free slot.
7. The system does not store historical parking data.

17. FUTURE SCOPE

The project can be improved significantly in future versions.

17.1 Ultrasonic Sensors

Ultrasonic sensors can replace the switches to automatically detect the presence of vehicles.

17.2 Automatic Gate

A servo motor can be added to control an automatic entry and exit barrier.

17.3 RFID

RFID technology can be used to identify registered vehicles.

17.4 IoT Connectivity

An ESP8266 or ESP32 can be used to send parking information to a cloud server or mobile application.

17.5 Mobile Application

A mobile application can show:

- Total parking capacity
- Available spaces
- Occupied spaces
- Individual slot status

17.6 Larger Parking Areas

The design can be expanded from four slots to dozens or hundreds of parking spaces.

17.7 Automatic Slot Guidance

LED indicators can guide drivers directly toward available parking spaces.

18. APPLICATIONS

This type of system can be used as a prototype for:

- Shopping mall parking
- College and university parking
- Office parking
- Hospital parking
- Apartment parking
- Hotel parking
- Public parking facilities
- Small commercial parking areas

19. CONCLUSION

The **Smart Parking System Using Arduino** successfully demonstrates the basic principles of automated parking management.

The Arduino continuously monitors four parking slots and determines whether each slot is occupied or vacant. Red and green LEDs provide an immediate visual indication of the status of each slot, while the 16×2 I2C LCD displays the total number of vacant spaces.

The system also provides detailed information through the Serial Monitor, making it useful for testing and debugging.

Although the current project is a small prototype, it provides a strong foundation for developing a larger smart parking system. By replacing the switches with ultrasonic or other vehicle-detection sensors and adding IoT connectivity, RFID, automatic gates, and mobile applications, the project can be transformed into a practical intelligent parking solution.

20. REFERENCES

1. Arduino programming and hardware documentation.
2. Arduino IDE documentation.
3. 16×2 LCD interfacing concepts.
4. I2C communication concepts.
5. Embedded-system programming concepts.
6. Digital input/output and pull-up resistor concepts.

APPENDIX A – COMPLETE SOURCE CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// =====
// I2C LCD
// =====
LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====
// SWITCHES / PARKING SENSORS
// =====
const int SLOT1 = A0;
const int SLOT2 = A1;
const int SLOT3 = A2;
const int SLOT4 = A3;

// =====
// RED LEDs
// =====
const int RED1 = 6;
const int RED2 = 8;
```

```

const int RED3 = 10;
const int RED4 = 12;

// =====
// GREEN LEDs
// =====
const int GREEN1 = 7;
const int GREEN2 = 9;
const int GREEN3 = 11;
const int GREEN4 = 13;

void setup()
{
  // Switch inputs
  pinMode(SLOT1, INPUT_PULLUP);
  pinMode(SLOT2, INPUT_PULLUP);
  pinMode(SLOT3, INPUT_PULLUP);
  pinMode(SLOT4, INPUT_PULLUP);

  // LED outputs
  pinMode(RED1, OUTPUT);
  pinMode(GREEN1, OUTPUT);

  pinMode(RED2, OUTPUT);
  pinMode(GREEN2, OUTPUT);

  pinMode(RED3, OUTPUT);
  pinMode(GREEN3, OUTPUT);

  pinMode(RED4, OUTPUT);
  pinMode(GREEN4, OUTPUT);

  // LCD
  lcd.init();
  lcd.backlight();

  // Serial Monitor
  Serial.begin(9600);

  lcd.setCursor(0, 0);
  lcd.print("SMART PARKING");

  lcd.setCursor(0, 1);
  lcd.print("System Ready");

  delay(2000);
  lcd.clear();
}

void loop()
{
  bool slot1Occupied = (digitalRead(SLOT1) == LOW);
  bool slot2Occupied = (digitalRead(SLOT2) == LOW);
  bool slot3Occupied = (digitalRead(SLOT3) == LOW);
  bool slot4Occupied = (digitalRead(SLOT4) == LOW);

```

```
if (slot1Occupied)
{
    digitalWrite(RED1, HIGH);
    digitalWrite(GREEN1, LOW);
}
else
{
    digitalWrite(RED1, LOW);
    digitalWrite(GREEN1, HIGH);
}
```

```
if (slot2Occupied)
{
    digitalWrite(RED2, HIGH);
    digitalWrite(GREEN2, LOW);
}
else
{
    digitalWrite(RED2, LOW);
    digitalWrite(GREEN2, HIGH);
}
```

```
if (slot3Occupied)
{
    digitalWrite(RED3, HIGH);
    digitalWrite(GREEN3, LOW);
}
else
{
    digitalWrite(RED3, LOW);
    digitalWrite(GREEN3, HIGH);
}
```

```
if (slot4Occupied)
{
    digitalWrite(RED4, HIGH);
    digitalWrite(GREEN4, LOW);
}
else
{
    digitalWrite(RED4, LOW);
    digitalWrite(GREEN4, HIGH);
}
```

```
int occupied = 0;
```

```
if (slot1Occupied)
    occupied++;
```

```
if (slot2Occupied)
    occupied++;
```

```
if (slot3Occupied)
    occupied++;
```

```
if (slot4Occupied)
```

```
    occupied++;

    int available = 4 - occupied;

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Vacant slots: ");
    lcd.print(available);

    lcd.setCursor(0, 1);

    if (available == 0)
    {
        lcd.print("PARKING FULL");
    }
    else
    {
        lcd.print("Thank You");
    }

    Serial.println("-----");

    Serial.print("Slot 1: ");
    Serial.println(slot1Occupied ? "OCCUPIED" : "EMPTY");

    Serial.print("Slot 2: ");
    Serial.println(slot2Occupied ? "OCCUPIED" : "EMPTY");

    Serial.print("Slot 3: ");
    Serial.println(slot3Occupied ? "OCCUPIED" : "EMPTY");

    Serial.print("Slot 4: ");
    Serial.println(slot4Occupied ? "OCCUPIED" : "EMPTY");

    Serial.print("Available Slots: ");
    Serial.println(available);

    delay(1000);
}
```

END OF REPORT